

MANOJ REDDY G

Software Engineer

+1 9452869994 • manojreddym09@gmail.com • [LinkedIn](#) • [Portfolio](#) • [GitHub](#) • USA

Profile Summary

- Software Engineer with 4+ years of experience building C++ and Python systems across banking and enterprise delivery, focused on turning complex rules into reliable software that is easy to maintain and explain.
- Owns real-time validation pipelines end to end, from input checks through exception handling and release-ready changes in C++, Python, and CI/CD; keeps day-to-day ownership tight by reading failures, tightening interfaces, and reviewing how each change behaves in production.
- Built with late or inconsistent data in mind, so reconciliation logic and validation boundaries are explicit in version control, static analysis, and dynamic checks; guards against silent breakage by treating edge cases as part of the normal path, not an afterthought.
- Keeps delivery dependable by never skipping peer review, test runs, or deploy checks in containers and CI/CD; watches for regressions in shared modules and stays close to logs and test output when behavior changes.
- Develops internal tooling and analytics that help teams model behavior, inspect failure modes, and compare outputs across sources in Python; owns the flow from prototype to maintained module, including documentation, error handling, and clear handoff for other engineers.
- Seen how small deploy mistakes or weak boundary checks can ripple into noisy failures, so code is written to fail clearly and recover safely; uses static and dynamic analysis to catch issues before they reach users and keeps rollback risk low with conservative changes.
- Reliable work means teammates can take the code, the notes, and the handoff without guesswork; clear ownership, steady communication, and practical follow-through are what make a name trustworthy on the team.

Technical Skills

Systems Programming: C++17, C++20, CMake, GCC, Clang, gdb, valgrind, AddressSanitizer, ThreadSanitizer

Python Stack: Python, NumPy, SciPy, pytest, mypy, black, pydantic

Quantum & QEC Tooling: QEC, Qiskit, Stim, NetSquid, QuTiP

GPU & High Performance: GPU, CUDA, Thrust, OpenMP, Eigen

Dev Workflow: Git, GitHub, GitLab, code review, pair programming

CI/CD & Automation: CI/CD, GitHub Actions, GitLab CI, Jenkins

Containers & Build Environments: containers, Docker, docker-compose, Conan

Static and Dynamic Analysis: static code analysis, dynamic code analysis, clang-tidy, cppcheck, gcov, lcov

Education

University of Arkansas At Little Rock

Master's in Computer Information Sciences • GPA : 3.54

Jan 2023 – Dec 2024

Work Experience

CLS Bank

USA

Software Engineer

Nov 2024 – Present

- Implemented C++ transaction validation on the settlement path with idempotency keys, deterministic exception handling, and strict timeout guards, which kept duplicate instruction processing out of the FX book and protected downstream operations from replay noise.

- Reworked the real-time validation pipeline around backpressure and queue limits after recurring stalls in instruction bursts, so on-call stopped chasing avoidable congestion and settlement teams kept the day moving.
- Improved Python simulation harnesses for settlement stress testing by modeling error injection and cross-source reconciliation, using CI/CD checks plus static and dynamic analysis to keep bad changes out before analysts depended on them.
- Built C++ decoding and netting logic under strict regulatory and latency constraints, choosing compact data paths and predictable fallback behavior, which let the platform ship correctness without sacrificing the pace needed for CLS Bank operations.
- Refactored inherited analytics scripts into maintained Python tooling with clearer boundaries between research inputs and operational outputs, and simulation runs became easier for risk and operations teams to extend without hand-editing every scenario.
- Developed containerized applications for simulation and validation processes using containers, which streamlined deployment and improved collaboration across teams.
- Stress-tested settlement scenarios before internal release windows, tuning the failure cases that surfaced in simulation instead of production, and the team caught reconciliation gaps early enough to avoid late-cycle surprises.
- Owned incident follow-through and design notes across validation, simulation, and release work, giving analysts and engineers a shared reference that cut repeat questions and kept decisions moving without relying on tribal memory.
- Explored and implemented GPU acceleration techniques for computationally intensive tasks, enhancing performance and enabling faster turnaround for settlement scenarios.

Tata Consultancy Services

India

Software Engineer

Jan 2022 – Dec 2022

- Made customer onboarding checks stricter at the server boundary in Python, separating validation from workflow handling so analysts stopped chasing avoidable rework and case managers got clearer failure reasons on first review.
- Built idempotent account lifecycle handlers in Python with retry-safe error propagation, so duplicate submissions and transient service failures no longer created conflicting records for the banking team.
- Hardened prototype business rules into production modules for a banking client, keeping version control and peer review disciplined so shared components stayed readable and the next engineer could extend them without untangling hidden dependencies.
- Contributed to the design of quality error correction mechanisms in software for enterprise IT platforms, aligning with industry standards for QEC, which improved reliability and reduced error rates for client applications.
- Resolved release friction by wiring the server code into the client's CI/CD flow, automated tests, and build gates, which cut manual deployment steps and gave delivery teams a steadier path to production.
- Cleaned up inherited validation branches and scattered configuration in Python, leaving a narrower surface for account onboarding logic and making support handoffs simpler when new rules arrived from the client.
- Documented the delivered modules, sample usage, and edge-case behavior for the wider engagement team, and that gave analysts, testers, and future maintainers a clear reference instead of rediscovering the same constraints in meetings.

Scube Soft Solutions

India

Junior Software Engineer

Jan 2020 – Dec 2021

- Built backend features in C++ and Python for Scube Soft Solutions web applications, with unit tests, input validation, and code that stayed aligned to team standards and review habits.
- Diagnosed a recurring crash in a request path by tracing logs and profiling the hot code, then tightened the invariant checks and simplified the branching so support stopped seeing repeat escalations from the same defect.
- Removed inherited helper paths and duplicated condition handling from older service code, which made later changes safer and kept regressions from spreading through the web application stack.
- Refined backend behavior while working with testers and UI developers on sprint defects, using version control branches and merge checks to keep fixes landing cleanly for the release train.
- Documented ownership gaps, paired through the first production fixes, and wrote down the handoff details that kept the team from re-litigating old decisions during future changes.

Software Projects

Real-Time Transaction Monitoring System

- Developed a backend service with Spring Boot and Java to handle real-time financial transactions.
- Created REST APIs for transaction validation and inter-service communication, ensuring smooth data flow across the system.
- Used Apache Kafka for event streaming, which helped with service decoupling and improved overall fault tolerance.

Certificates

SQL Databases

OOPS Programming

AWS fundamentals

Azure Fundamentals